

Tema 02 · Singleton, DAO con transacciones y Maven multimódulo

Clase 016 (05-08) · Teoría: «Base de datos – Parte III» y «Patrones de diseño»

Programación Avanzada · Prof. Macrino Matías · Universidad del Salvador – Sede Pilar
· 2.º cuatrimestre

1. Patrones de diseño

Un **patrón de diseño** es el «esqueleto de la solución» a un problema que aparece una y otra vez en el desarrollo de software. Para ser un patrón debe haber demostrado su efectividad y poder reutilizarse. Se clasifican en:

- **Creacionales:** controlan cómo se crean los objetos (Singleton, Factory, DAO en la clasificación de la cátedra).
- **Estructurales:** cómo se agrupan clases y objetos en estructuras mayores, separando interfaz de implementación.
- **De comportamiento:** cómo se comunican los objetos entre sí.

2. Patrón Singleton

Garantiza que una clase tenga una única instancia y proporciona un punto de acceso global a ella. Se usa cuando un recurso debe gestionarse de forma centralizada: la configuración de la aplicación, el pool/conexión a la base de datos, un log. Las **tres reglas** que hay que saber:

Regla	Cómo se implementa
1. Constructor privado	Nadie fuera de la clase puede hacer new.
2. Atributo estático que guarda la instancia	private static Clase instancia; — pertenece a la clase, no a un objeto.
3. Método público estático de acceso	getInstance(): si instancia es null la crea; siempre devuelve la misma.

Hilos: en una aplicación web varios hilos pueden llamar a getInstance() a la vez; si dos ven instancia == null al mismo tiempo se crearían dos objetos. Por eso se usa synchronized en el método (como en la clase) o una inicialización segura (holder idiom, enum). Prueba típica: pedir la instancia dos veces y comparar con ==: son el mismo objeto.

3. Patrón DAO (Data Access Object)

Abstrae el acceso a datos para que la lógica de negocio no sepa cómo ni dónde se guardan los datos. Tiene tres piezas:

Pieza	Qué es	En esta clase
Entidad (POJO / DTO)	Clase simple con atributos, constructores, getters y setters que representa un dato del negocio	Cuenta, Movimiento
Interfaz DAO	Contrato con las operaciones (CRUD: Create, Read, Update, Delete) sin mencionar la tecnología	CuentaDAO, MovimientoDAO
Implementación (DAOImpl)	La lógica concreta (JDBC, memoria, archivos...)	CuentaDAOImpl, MovimientoDAOImpl

Junto con el patrón **Factory** (una clase que decide qué implementación entregar) se puede cambiar de persistencia en memoria a JDBC sin tocar el código que usa el DAO. Además suele haber una **capa de**

servicio (acá TransferenciaService) que coordina varios DAO para implementar un caso de uso.

4. Transacciones JDBC

Una **transacción** es un conjunto de operaciones que debe ejecutarse como una unidad **atómica**: o se hacen todas o no se hace ninguna. El ejemplo clásico es la **transferencia bancaria**: restar a la cuenta A y sumar a la cuenta B. Si se resta pero falla el sumar, el dinero desaparece. En JDBC:

```
con.setAutoCommit(false); // desactiva el commit automático: empieza la transacción
try {
    ... sentencias SQL ...
    con.commit();           // todo salió bien: se confirman los cambios
} catch (SQLException e) {
    con.rollback();         // algo falló: se deshace todo
} finally {
    con.close();            // siempre se libera la conexión
}
```

Punto clave: todas las sentencias de la transacción deben usar **la misma Connection**. Por eso, en el DAO, los métodos que forman parte de una transacción **reciben la conexión por parámetro** (descontarSaldo(connection, cuenta)) y es el **servicio** quien abre la conexión, controla commit y rollback y la cierra.

5. Maven: ciclo de vida y multimódulo

Maven automatiza la construcción del proyecto y la gestión de dependencias. Todo se declara en el **pom.xml** (Project Object Model). Las fases del ciclo de vida son secuenciales:

Fase	Qué hace
clean	Borra la carpeta target (compilaciones anteriores)
compile	Compila el código fuente a bytecode
test	Ejecuta las pruebas unitarias
package	Empaqueta en JAR o WAR
install	Publica el artefacto en el repositorio local (~/.m2) para usarlo desde otros proyectos

Un **Maven Parent** (packaging pom) agrupa varios **módulos**. Cada módulo hijo declara su <parent> y puede depender de otro módulo como si fuera una librería. Es exactamente la estructura que pide el parcial: un proyecto **DAO** (persistencia) y un proyecto **Web/MVC** (presentación) que depende del DAO.

6. Excepciones propias

NegocioException extiende de **RuntimeException** (no controlada): no obliga a declararla ni capturarla. Se usa para errores de regla de negocio (saldo insuficiente). Contraste: **SQLException** es checked (controlada) y obliga a tratarla. Elegir una u otra es una decisión de diseño que conviene poder justificar.

Código de las clases, explicado línea por línea

Cada recuadro gris reproduce líneas reales del archivo (con su número de línea) y el recuadro verde de abajo explica qué hacen y por qué están escritas así.

Clase 016 - 05-08: Singleton y transferencia bancaria con Maven multimódulo

La carpeta contiene un **Maven parent** con dos módulos: 016 - Clase 05-08 (la capa de datos: dominio, DAO, servicio de transferencia y excepción) y 016 - Clase 05-08 single-maven (un ejemplo de Singleton que **depende** del primer módulo). El código de la transferencia es un **esqueleto de clase**: muestra la estructura y el flujo transaccional pero algunos pasos están marcados con TODO o vacíos. Más abajo se señalan esas partes.

016 - Clase 05-08 Maven Parent / pom.xml

Qué es y para qué sirve: POM **padre** (packaging pom): agrupa los dos módulos de la clase.

Cómo defenderlo / qué decir: Con un padre se compilan todos los módulos con un solo comando (mvn package en la raíz) y se comparte la configuración (Java 11, UTF-8). Maven ordena solo la compilación según las dependencias entre módulos.

```
1 | <?xml version="1.0" encoding="UTF-8"?>
2 | <project xmlns="http://maven.apache.org/POM/4.0.0"
3 |       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4 |       xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5 |   <modelVersion>4.0.0</modelVersion>
6 |
```

Líneas 1 a 6: Cabecera del POM.

```
7 |   <groupId>edu.usal</groupId>
8 |   <artifactId>clase-05-08-parent</artifactId>
9 |   <version>1.0-SNAPSHOT</version>
10 |   <packaging>pom</packaging>
11 |
```

Líneas 7 a 11: Coordenadas del padre y **<packaging>pom</packaging>**: no genera jar ni war, solo agrupa.

```
12 |   <properties>
13 |     <maven.compiler.source>11</maven.compiler.source>
14 |     <maven.compiler.target>11</maven.compiler.target>
15 |     <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
16 |   </properties>
17 |
```

Líneas 12 a 17: Java 11 y UTF-8 para todos los módulos hijos (se heredan).

```
18 |   <modules>
19 |     <module>016 - Clase 05-08</module>
20 |     <module>016 - Clase 05-08 single-maven</module>
21 |   </modules>
22 |
23 | </project>
```

Líneas 18 a 23: Lista de **módulos** hijos: el de datos y el del Singleton.

016 - Clase 05-08 Maven Parent / 016 - Clase 05-08 / pom.xml

Qué es y para qué sirve: POM del módulo de datos (hijo del padre), con la dependencia del driver de MySQL.

Cómo defenderlo / qué decir: Hereda las propiedades del padre mediante **<parent>**. Es el módulo que luego usa el otro como dependencia.

```

1 | <?xml version="1.0" encoding="UTF-8"?>
2 | <project xmlns="http://maven.apache.org/POM/4.0.0"
3 |     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4 |     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5 |     <modelVersion>4.0.0</modelVersion>
6 |

```

Líneas 1 a 6: Cabecera.

```

7 |     <parent>
8 |         <groupId>edu.usal</groupId>
9 |         <artifactId>clase-05-08-parent</artifactId>
10 |        <version>1.0-SNAPSHOT</version>
11 |     </parent>
12 |
13 |     <groupId>edu.usal</groupId>

```

Líneas 7 a 13: Declara el **padre** (mismas coordenadas que el pom padre): de ahí hereda propiedades.

```

14 |         <artifactId>clase-05-08</artifactId>
15 |         <version>1.0-SNAPSHOT</version>
16 |

```

Líneas 14 a 16: Coordenadas propias del módulo: **clase-05-08**.

```

17 |     <dependencies>
18 |         <dependency>
19 |             <groupId>com.mysql</groupId>
20 |             <artifactId>mysql-connector-j</artifactId>
21 |             <version>9.2.0</version>
22 |         </dependency>
23 |     </dependencies>
24 |

```

Líneas 17 a 24: Depende del driver JDBC de MySQL 9.2.0.

```

25 | </project>

```

Línea 25: Cierre.

016 - Clase 05-08 Maven Parent / 016 - Clase 05-08 single-maven / pom.xml

Qué es y para qué sirve: POM del módulo Singleton. **Depende del módulo clase-05-08.**

Cómo defenderlo / qué decir: Demuestra la dependencia entre módulos: para usarlo, clase-05-08 tiene que estar compilado o instalado (mvn install). Es el mismo mecanismo con el que el módulo Web del parcial usa al módulo DAO.

```

1 | <?xml version="1.0" encoding="UTF-8"?>
2 | <project xmlns="http://maven.apache.org/POM/4.0.0"
3 |     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4 |     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5 |     <modelVersion>4.0.0</modelVersion>
6 |

```

Líneas 1 a 6: Cabecera.

```

7 |     <parent>
8 |         <groupId>edu.usal</groupId>
9 |         <artifactId>clase-05-08-parent</artifactId>
10 |        <version>1.0-SNAPSHOT</version>
11 |     </parent>
12 |
13 |     <groupId>edu.usal</groupId>

```

Líneas 7 a 13: Declara el mismo padre.

```

14 |         <artifactId>clase-05-08-single-maven</artifactId>
15 |         <version>1.0-SNAPSHOT</version>
16 |
17 |
18 |

```

Líneas 14 a 18: Coordenadas del módulo.

```

19 |     <dependencies>
20 |         <dependency>
21 |             <groupId>edu.usal</groupId>
22 |             <artifactId>clase-05-08</artifactId>
23 |             <version>1.0-SNAPSHOT</version>
24 |         </dependency>
25 |     </dependencies>
26 |

```

Líneas 19 a 26: Dependencia de **otro módulo del mismo proyecto** (clase-05-08), tratado igual que una librería externa.

```

27 | </project>

```

Línea 27: Cierre.

016 - Clase 05-08 Maven Parent / 016 - Clase 05-08 single-maven / src / main / java / edu / usal / singleton / ConfigSingleton.java

Qué es y para qué sirve: Implementación del patrón **Singleton**: una configuración global de la aplicación (idioma, versión, tema).

Cómo defenderlo / qué decir: Cumple las reglas del patrón: atributo estático instancia y método estático getInstance(); **synchronized** evita que dos hilos creen dos instancias. **Observación honesta:** el constructor está declarado public; para ser un Singleton estricto debería ser **private** (regla 1 del propio comentario de la clase), porque con constructor público cualquiera puede hacer new ConfigSingleton() y romper la unicidad.

```

1 | package edu.usal.singleton;
2 |

```

Líneas 1 a 2: Paquete singleton.

```

3 | import java.util.HashMap;
4 | import java.util.Map;
5 |

```

Líneas 3 a 5: Importaciones: **HashMap, Map**. Cada import permite usar esa clase por su nombre corto en lugar de escribir el paquete completo.

```

6 | /*
7 |     Patron Singleton : Garantizar que exista una unica instancia y de acceso de global
8 |
9 |     Reglas del patron:
10 |
11 |     1 - Constructor privado
12 |     2 - Atributo de clase estatico
13 |     3 - Metodo publico y estatico de acceso "getInstance()"
14 | */

```

Líneas 6 a 14: Comentario de la clase: define el patrón y sus **tres reglas**: constructor privado, atributo de clase estático y método público estático getInstance().

```

15 | public class ConfigSingleton {
16 |

```

Líneas 15 a 16: Declara la clase.

```

17 |     private static ConfigSingleton instancia;
18 |     private boolean flag;
19 |
20 |     private Map<String, String> propiedades;
21 |

```

Líneas 17 a 21: Atributo **estático**: guarda la única instancia (vale null hasta que se pide la primera vez). Además un flag y un **Map** de propiedades clave → valor.

```

22 |     public ConfigSingleton() {
23 |         this.propiedades = new HashMap<>();
24 |
25 |         this.propiedades.put("idioma", "ES");
26 |         this.propiedades.put("version", "1.0.0");
27 |         this.propiedades.put("tema", "Oscuro");
28 |     }
29 |

```

Líneas 22 a 29: Constructor: crea el mapa y carga tres propiedades por defecto (idioma ES, versión 1.0.0, tema Oscuro). Debería ser privado.

```

30 |     public static synchronized ConfigSingleton getInstance(){
31 |         if(instancia == null){
32 |             instancia = new ConfigSingleton();
33 |         }
34 |
35 |         return instancia;
36 |     }
37 |

```

Líneas 30 a 37: Método de acceso global. **static:** se llama sobre la clase. **synchronized:** un hilo a la vez, así no se crean dos instancias. Si instancia es null (primer uso) la crea (**inicialización perezosa**, lazy) y siempre devuelve la misma.

```

38 |     /*public Map<String, String> getPropiedades() {
39 |         return propiedades;
40 |     }*/
41 |

```

Líneas 38 a 41: Getter del mapa comentado: se evita exponer la colección interna (encapsulamiento).

```

42 |     public void setPropiedades(Map<String, String> propiedades) {
43 |         this.propiedades = propiedades;
44 |     }
45 |

```

Líneas 42 a 45: Reemplaza todas las propiedades.

```

46 |     public String getPropiedad(String key){
47 |         return this.propiedades.get(key);
48 |     }
49 |

```

Líneas 46 a 49: Lee una propiedad por su clave.

```

50 |     public void setPropiedad(String key, String valor){
51 |         this.propiedades.put(key, valor);
52 |     }
53 | }

```

Líneas 50 a 53: Agrega o modifica una propiedad.

016 - Clase 05-08 Maven Parent / 016 - Clase 05-08 single-maven / src / main / java / edu / usal / singleton / PruebaSingleton.java

Qué es y para qué sirve: main que demuestra que el Singleton devuelve siempre el mismo objeto.

Cómo defenderlo / qué decir: La prueba clásica: pedir la instancia dos veces y comparar referencias con == (no con equals). Si son iguales, es una única instancia.

```

1 | package edu.usal.singleton;
2 |

```

Líneas 1 a 2: Paquete.

```

3 | public class PruebaSingleton {
4 |

```

Líneas 3 a 4: Clase de prueba.

```

5 |     public static void main(String[] args) {
6 |         ConfigSingleton config1 = ConfigSingleton.getInstance();
7 |         ConfigSingleton config2 = ConfigSingleton.getInstance();
8 |     }

```

Líneas 5 a 8: Pide la instancia dos veces con `ConfigSingleton.getInstance()` (nunca con `new`).

```

9 |         if(config1 == config2){
10 |             System.out.println("Instancias Iguales");
11 |             System.out.println("Config1 " + config1);
12 |             System.out.println("Config2 " + config2);
13 |         } else {
14 |             System.out.println("Instancias distintas");
15 |         }
16 |     }
17 | }

```

Líneas 9 a 17: Compara las **referencias**: si apuntan al mismo objeto imprime «Instancias Iguales» (con su representación) y si no, «Instancias distintas».

016 - Clase 05-08 Maven Parent / 016 - Clase 05-08 / src / main / java / edu / usal / basedatos / util / MySqlConnectionor.java

Qué es y para qué sirve: Fábrica de conexiones a MySQL (igual a la del Tema 01 pero en el paquete `util`).

Cómo defenderlo / qué decir: Ver el Tema 01. Acá la usan los DAO y el servicio. Es el candidato natural a convertirse en **Singleton de conexión** (la `ConexionBD` de la guía de estudio): constructor privado, instancia estática y `getInstance()`.

```

1 | package edu.usal.basedatos.util;
2 |

```

Líneas 1 a 2: Paquete `util`.

```

3 | import java.sql.Connection;
4 | import java.sql.DatabaseMetaData;
5 | import java.sql.DriverManager;
6 | import java.sql.SQLException;
7 |

```

Líneas 3 a 7: Importaciones: **Connection**, **DatabaseMetaData**, **DriverManager**, **SQLException**. Cada `import` permite usar esa clase por su nombre corto en lugar de escribir el paquete completo.

```

8 | public class MySqlConnectionor {
9 |

```

Líneas 8 a 9: Clase conectora.

```

10 |     public static Connection getConnection(){
11 |         try {

```

Líneas 10 a 11: Devuelve una **Connection** nueva.

```

12 |             Class.forName("com.mysql.cj.jdbc.Driver");
13 |         } catch (ClassNotFoundException e) {
14 |             System.out.println(e.getMessage());
15 |         }
16 |     }

```

Líneas 12 a 16: Carga el driver de MySQL.

```

17 |         String urlJdbc = "jdbc:mysql://localhost:3306/universidad";
18 |         String user = "root";
19 |         String password = "123456";
20 |         Connection connection = null;
21 |
22 |         try {

```

Líneas 17 a 22: URL, usuario y contraseña de la base.

```

23 |         connection = DriverManager.getConnection(urlJdbc,
24 |             user,
25 |             password);
26 |
27 |

```

Líneas 23 a 27: Abre la conexión.

```

28 |         } catch (SQLException e) {
29 |             System.out.println(e.getMessage());
30 |         }
31 |

```

Líneas 28 a 31: Informa el error.

```

32 |         return connection;
33 |     }
34 |
35 | }

```

Líneas 32 a 35: Devuelve la conexión.

016 - Clase 05-08 Maven Parent / 016 - Clase 05-08 / src / main / java / edu / usal / basedatos / domain / Cuenta.java

Qué es y para qué sirve: Entidad (POJO) de una **cuenta bancaria**: número, saldo y cliente.

Cómo defenderlo / qué decir: Primera pieza del patrón DAO: una clase de datos sin lógica. Atributos **privados** con getters/setters (encapsulamiento) y constructor vacío (necesario para crear el objeto y completarlo con setters). Usa los wrappers Integer y Double que admiten null (el id no existe antes de guardarse).

```

1 | package edu.usal.basedatos.domain;
2 |

```

Líneas 1 a 2: Paquete domain (dominio).

```

3 | public class Cuenta {
4 |
5 |     private Integer numeroCuenta;
6 |     private Double saldo;
7 |     private Integer idCliente;
8 |

```

Líneas 3 a 8: Atributos privados: número de cuenta, saldo e id del cliente.

```

9 |     public Cuenta() {
10 |     }
11 |

```

Líneas 9 a 11: Constructor vacío.

```

12 |     public Double getSaldo() {
13 |         return saldo;
14 |     }
15 |
16 |     public void setSaldo(Double saldo) {
17 |         this.saldo = saldo;
18 |     }
19 |
20 |     public Integer getIdCliente() {
21 |         return idCliente;
22 |     }
23 |
24 |     public void setIdCliente(Integer idCliente) {
25 |         this.idCliente = idCliente;
26 |     }
27 |
28 |     public Integer getNumeroCuenta() {
29 |         return numeroCuenta;
30 |     }
31 |
32 |     public void setNumeroCuenta(Integer numeroCuenta) {
33 |         this.numeroCuenta = numeroCuenta;
34 |     }
35 | }

```

Líneas 12 a 35: Getters y setters: métodos públicos que leen (get) y modifican (set) los atributos privados. Es el **encapsulamiento**: nadie toca los datos directamente.

016 - Clase 05-08 Maven Parent / 016 - Clase 05-08 / src / main / java / edu / usal / basedatos / domain / Moviento.java

Qué es y para qué sirve: Entidad que registra un **movimiento** (una transferencia): cuenta origen, cuenta destino y fecha-hora.

Cómo defenderlo / qué decir: Sirve de comprobante de cada transferencia. Usa **LocalDateTime** (API java.time moderna) en lugar de Date. El nombre de la clase tiene un error de tipeo («Moviento»); conviene mencionarlo si se pregunta, se corrige renombrando a Movimiento.

```

1 | package edu.usal.basedatos.domain;
2 |

```

Líneas 1 a 2: Paquete domain.

```

3 | import java.time.LocalDateTime;
4 |

```

Líneas 3 a 4: Importaciones: **LocalDateTime**. Cada import permite usar esa clase por su nombre corto en lugar de escribir el paquete completo.

```

5 | public class Moviento {
6 |
7 |     private String cuentaOrigen;
8 |     private String cuentaDestino;
9 |     private LocalDateTime fechaHora;
10 |

```

Líneas 5 a 10: Atributos privados: cuentaOrigen, cuentaDestino (texto) y fechaHora (**LocalDateTime**).

```

11 |     public Moviento() {
12 |     }
13 |

```

Líneas 11 a 13: Constructor vacío.

```

14 |     public String getCuentaOrigen() {
15 |         return cuentaOrigen;
16 |     }
17 |
18 |     public void setCuentaOrigen(String cuentaOrigen) {
19 |         this.cuentaOrigen = cuentaOrigen;
20 |     }
21 |
22 |     public String getCuentaDestino() {
23 |         return cuentaDestino;
24 |     }
25 |
26 |     public void setCuentaDestino(String cuentaDestino) {
27 |         this.cuentaDestino = cuentaDestino;
28 |     }
29 |
30 |     public LocalDateTime getFechaHora() {
31 |         return fechaHora;
32 |     }
33 |
34 |     public void setFechaHora(LocalDateTime fechaHora) {
35 |         this.fechaHora = fechaHora;
36 |     }
37 | }

```

Líneas 14 a 37: Getters y setters: métodos públicos que leen (get) y modifican (set) los atributos privados. Es el **encapsulamiento**: nadie toca los datos directamente.

016 - Clase 05-08 Maven Parent / 016 - Clase 05-08 / src / main / java / edu / usal / basedatos / exception / NegocioException.java

Qué es y para qué sirve: Excepción propia para errores de **reglas de negocio** (por ejemplo, saldo insuficiente).

Cómo defenderlo / qué decir: Extiende **RuntimeException**: es no controlada, no obliga a usar throws/try. Separa los errores de negocio de los técnicos (SQLException). Está vacía: faltaría agregar un constructor con mensaje (super(mensaje)) para informar la causa.

```

1 | package edu.usal.basedatos.exception;
2 |

```

Líneas 1 a 2: Paquete exception.

```

3 | public class NegocioException extends RuntimeException{
4 |
5 |
6 | }

```

Líneas 3 a 6: Excepción propia heredando de RuntimeException (sin cuerpo).

016 - Clase 05-08 Maven Parent / 016 - Clase 05-08 / src / main / java / edu / usal / basedatos / dao / CuentaDAO.java

Qué es y para qué sirve: **Interfaz DAO** de las cuentas: el contrato de las operaciones.

Cómo defenderlo / qué decir: El contrato no menciona tecnologías (JDBC). Decisión de diseño importante, que se explica en el comentario del código: los métodos que forman parte de una **transacción** reciben la Connection por parámetro para que todos trabajen sobre la misma; validarSaldo es solo una consulta y no la necesita.

```

1 | package edu.usal.basedatos.dao;
2 |

```

Líneas 1 a 2: Paquete dao.

```

3 | import edu.usal.basedatos.domain.Cuenta;
4 | import edu.usal.basedatos.exception.NegocioException;
5 |
6 | import java.sql.Connection;
7 | import java.sql.SQLException;
8 |

```

Líneas 3 a 8: Importaciones: **Cuenta**, **NegocioException**, **Connection**, **SQLException**. Cada import permite usar esa clase por su nombre corto en lugar de escribir el paquete completo.

```
9 | public interface CuentaDAO {
10 |
```

Líneas 9 a 10: Interfaz: solo declara métodos, sin código.

```
11 | //Connection no se notifica en el contrato del metodo o accion
12 |
```

Líneas 11 a 12: Comentario del autor: la conexión no se declara en el contrato funcional pero aquí se pasa como parámetro para compartir la transacción.

```
13 | void descontarSaldo(Connection connection, Cuenta cuenta) throws SQLException;
```

Línea 13: Resta saldo de una cuenta, usando la conexión recibida (**throws SQLException**).

```
14 | void incrementarSaldo(Connection connection, Cuenta cuenta) throws SQLException;
```

Línea 14: Suma saldo a una cuenta.

```
15 | double validarSaldo(int numeroCuenta) throws NegocioException;
16 |
17 | }
```

Líneas 15 a 17: Consulta el saldo de una cuenta; puede lanzar **NegocioException**.

016 - Clase 05-08 Maven Parent / 016 - Clase 05-08 / src / main / java / edu / usal / basedatos / dao / MovimientoDAO.java

Qué es y para qué sirve: Interfaz DAO de los movimientos.

Cómo defenderlo / qué decir: Un único método que también recibe la conexión: el registro del movimiento forma parte de la misma transacción de la transferencia.

```
1 | package edu.usal.basedatos.dao;
2 |
```

Líneas 1 a 2: Paquete dao.

```
3 | import edu.usal.basedatos.domain.Movimiento;
4 |
5 | import java.sql.Connection;
6 |
```

Líneas 3 a 6: Importaciones: **Movimiento**, **Connection**. Cada import permite usar esa clase por su nombre corto en lugar de escribir el paquete completo.

```
7 | public interface MovimientoDAO {
8 |
9 |     void registrarMovimiento(Connection connection, Movimiento movimiento);
10 | }
```

Líneas 7 a 10: Interfaz con el método **registrarMovimiento(connection, movimiento)**.

016 - Clase 05-08 Maven Parent / 016 - Clase 05-08 / src / main / java / edu / usal / basedatos / dao / CuentaDAOImpl.java

Qué es y para qué sirve: Implementación **JDBC** de CuentaDAO con **PreparedStatement**.

Cómo defenderlo / qué decir: Aplica lo aprendido: SQL con **?**, setXxx y executeUpdate. Los UPDATE usan **saldo = saldo ± ?**: la suma/resta la hace la base en una sola operación (más seguro que leer, calcular en Java y escribir). **Observaciones de mejora que conviene conocer:** (1) los métodos no cierran el PreparedStatement; (2) quedan los TODO para validar las filas afectadas; (3) validarSaldo llama a getDouble sin resultSet.next() antes, lo que lanzaría una SQLException (el cursor está antes de la primera fila), y abre su propia conexión sin cerrarla.

```
1 | package edu.usal.basedatos.dao;
2 |
```

Líneas 1 a 2: Paquete dao.

```

3 | import edu.usal.basedatos.domain.Cuenta;
4 | import edu.usal.basedatos.exception.NegocioException;
5 | import edu.usal.basedatos.util.MySqlConector;
6 |
7 | import java.sql.Connection;
8 | import java.sql.PreparedStatement;
9 | import java.sql.ResultSet;
10 | import java.sql.SQLException;
11 |

```

Líneas 3 a 11: Importaciones: **Cuenta**, **NegocioException**, **MySqlConector**, **Connection**, **PreparedStatement**, **ResultSet**, **SQLException**. Cada import permite usar esa clase por su nombre corto en lugar de escribir el paquete completo.

```

12 | public class CuentaDAOImpl implements CuentaDAO{
13 |
14 |     private Connection connection;
15 |     private PreparedStatement pStatement;
16 |

```

Líneas 12 a 16: Implementa la interfaz. Guarda la conexión y el PreparedStatement como atributos.

```

17 |     @Override

```

Línea 17: **@Override** indica que se implementa un método de la interfaz.

```

18 |     public void descontarSaldo(Connection connection, Cuenta cuenta) throws SQLException {

```

Línea 18: Resta saldo. Usa la **conexión recibida** (la de la transacción).

```

19 |         String strSQL = "UPDATE cuentas set saldo = saldo - ? WHERE numero_cuenta = ?";

```

Línea 19: SQL con dos comodines: monto y número de cuenta. La resta se hace dentro de la base.

```

20 |         this.connection = connection;
21 |         this.pStatement = this.connection.prepareStatement(strSQL);

```

Líneas 20 a 21: Guarda la conexión y crea el **PreparedStatement** a partir del SQL.

```

22 |         this.pStatement.setDouble(1, cuenta.getSaldo());
23 |         this.pStatement.setInt(2, cuenta.getNumeroCuenta());

```

Líneas 22 a 23: Asigna el monto (en este diseño la «cuenta» lleva el monto a descontar en su saldo) y el número de cuenta.

```

24 |         int filasAfectadas = this.pStatement.executeUpdate();
25 |         //TODO: validar filas afectadas
26 |     }
27 |
28 |     @Override

```

Líneas 24 a 28: Ejecuta el UPDATE y guarda las filas afectadas; el **TODO** recuerda que falta validar que haya sido 1 (si la cuenta no existe, no se modifica nada y no se avisa).

```

29 |     public void incrementarSaldo(Connection connection, Cuenta cuenta) throws SQLException {
30 |         String strSQL = "UPDATE cuentas set saldo = saldo + ? WHERE numero_cuenta = ?";
31 |         this.connection = connection;
32 |         this.pStatement = this.connection.prepareStatement(strSQL);
33 |         this.pStatement.setDouble(1, cuenta.getSaldo());
34 |         this.pStatement.setInt(2, cuenta.getNumeroCuenta());
35 |         int filasAfectadas = this.pStatement.executeUpdate();
36 |         //TODO: validar filas afectadas
37 |     }
38 |
39 |     @Override

```

Líneas 29 a 39: Igual que el anterior pero **suma** el monto a la cuenta destino.

```

40 |     public double validarSaldo(int numeroCuenta) throws NegocioException {
41 |         String strSQL = "SELECT saldo FROM cuentas WHERE numero_cuenta = ?";
42 |         this.connection = MySqlConector.getConnection();
43 |         try {

```

Líneas 40 a 43: Consulta el saldo de una cuenta. Esta vez **abre su propia conexión** (es solo lectura y no forma parte de la transacción).

```

44 |         this.pStatement = this.connection.prepareStatement(strSQL);
45 |         this.pStatement.setInt(1, numeroCuenta);
46 |         ResultSet resultSet = this.pStatement.executeQuery();

```

Líneas 44 a 46: Prepara el SELECT, asigna el número de cuenta y ejecuta con **executeQuery()**.

```

47 |         return resultSet.getDouble("saldo");

```

Línea 47: Lee el saldo. Falta **resultSet.next()** antes de leer.

```

48 |         } catch (SQLException e) {
49 |             throw new RuntimeException(e);
50 |         }
51 |     }
52 | }
53 | }

```

Líneas 48 a 53: Convierte la **SQLException** en una **RuntimeException** (se «envuelve» para no obligar a capturarla).

016 - Clase 05-08 Maven Parent / 016 - Clase 05-08 / src / main / java / edu / usal / basedatos / dao / MovimientoDAOImpl.java

Qué es y para qué sirve: Implementación (vacía) de **MovimientoDAO**.

Cómo defenderlo / qué decir: Quedó como esqueleto: muestra dónde iría el INSERT del movimiento usando la conexión recibida (para que forme parte de la transacción).

```

1 | package edu.usal.basedatos.dao;
2 |

```

Líneas 1 a 2: Paquete **dao**.

```

3 | import edu.usal.basedatos.domain.Moviento;
4 |
5 | import java.sql.Connection;
6 |

```

Líneas 3 a 6: Importaciones: **Moviento**, **Connection**. Cada import permite usar esa clase por su nombre corto en lugar de escribir el paquete completo.

```

7 | public class MovimientoDAOImpl implements MovimientoDAO{

```

Línea 7: Implementa la interfaz.

```

8 |     @Override
9 |     public void registrarMovimiento(Connection connection, Moviento moviento) {
10 |
11 |     }
12 | }

```

Líneas 8 a 12: Método **registrarMovimiento** con el cuerpo vacío: pendiente de implementar (INSERT con **PreparedStatement** sobre la conexión recibida).

016 - Clase 05-08 Maven Parent / 016 - Clase 05-08 / src / main / java / edu / usal / basedatos / services / TransferenciaService.java

Qué es y para qué sirve: **Capa de servicio**: orquesta los DAO y controla la **transacción** de una transferencia bancaria.

Cómo defenderlo / qué decir: Es la clase más importante de la clase: muestra el patrón completo **setAutoCommit(false) → operaciones → commit / rollback → close**. El servicio abre la conexión y la pasa a cada DAO para que todos trabajen en la misma transacción. Si falla cualquier paso, **rollback()** deshace todo. **Observaciones:** es un esqueleto (**transferir()** sin parámetros, objetos vacíos); el **rollback** está en el catch pero el cierre de la conexión debería ir en un **finally**; y la **NegocioException** se atrapa sin hacer nada (en una versión real habría que abortar la transferencia).

```

1 | package edu.usal.basedatos.services;
2 |

```

Líneas 1 a 2: Paquete **services** (servicios).

```

3 | import edu.usal.basedatos.dao.CuentaDAO;
4 | import edu.usal.basedatos.dao.CuentaDAOImpl;
5 | import edu.usal.basedatos.dao.MovimientoDAO;
6 | import edu.usal.basedatos.dao.MovimientoDAOImpl;
7 | import edu.usal.basedatos.domain.Cuenta;
8 | import edu.usal.basedatos.domain.Moviento;
9 | import edu.usal.basedatos.exception.NegocioException;
10 | import edu.usal.basedatos.util.MySqlConector;
11 |
12 | import java.sql.Connection;
13 | import java.sql.SQLException;
14 |

```

Líneas 3 a 14: Importaciones: **CuentaDAO**, **CuentaDAOImpl**, **MovimientoDAO**, **MovimientoDAOImpl**, **Cuenta**, **Moviento**, **NegocioException**, **MySqlConector**, **Connection**, **SQLException**. Cada import permite usar esa clase por su nombre corto en lugar de escribir el paquete completo.

```

15 | public class TransferenciaService {
16 |
17 |     private CuentaDAO cuentaDAO;
18 |     private MovimientoDAO movimientoDAO;
19 |

```

Líneas 15 a 19: Atributos: los dos DAO, declarados con el tipo **interfaz** (se programa contra la interfaz).

```

20 |     public TransferenciaService() {
21 |         this.cuentaDAO = new CuentaDAOImpl();
22 |         this.movimientoDAO = new MovimientoDAOImpl();
23 |     }
24 |

```

Líneas 20 a 24: Constructor: crea las implementaciones concretas. Aquí es donde un **Factory** podría decidir qué implementación entregar.

```

25 |     public void transferir(){
26 |         Connection connection = MySqlConector.getConnection();
27 |         try {

```

Líneas 25 a 27: Método principal. Abre una conexión nueva.

```

28 |             connection.setAutoCommit(false);
29 |         } catch (SQLException e) {
30 |             throw new RuntimeException(e);
31 |         }
32 |
33 |         try{
34 |

```

Líneas 28 a 34: Inicia la transacción: a partir de acá nada se confirma hasta el commit.

```

35 |             //1er Paso - Validar saldo cuenta origen
36 |             this.cuentaDAO.validarSaldo(0);
37 |         }catch (NegocioException e){
38 |             //Manejo de exception
39 |         }
40 |
41 |         try {
42 |

```

Líneas 35 a 42: Paso 1: valida el saldo de la cuenta origen (consulta). Si hay **NegocioException** se capturaría para manejar el error (vacío en el esqueleto).

```

43 |             //2do Paso - DescontarSaldo cuenta origen
44 |             this.cuentaDAO.descontarSaldo(connection, new Cuenta());
45 |
46 |             //3er Paso - Incrementar cuenta destino
47 |             this.cuentaDAO.incrementarSaldo(connection, new Cuenta());
48 |
49 |             //4to Paso - Registrar movimiento
50 |             this.movimientoDAO.registrarMovimiento(connection, new Moviento());
51 |
52 |             connection.commit();
53 |

```

Líneas 43 a 53: Pasos 2, 3 y 4 dentro de un mismo try: descontar a origen, incrementar a destino y registrar el movimiento, todos con **la misma conexión**. Si todo sale bien, **commit()** confirma los cambios.

```

54 |         } catch (SQLException e){
55 |             //Manejo de excepcion
56 |             try {
57 |                 connection.rollback();
58 |             } catch (SQLException ex) {
59 |                 throw new RuntimeException(ex);
60 |             }
61 |         }
62 |     }
63 |     try {

```

Líneas 54 a 63: Si alguno de los pasos falla, **rollback()** deshace todo lo hecho en la transacción: ninguna cuenta queda modificada.

```

64 |         connection.close();
65 |     } catch (SQLException e) {
66 |         throw new RuntimeException(e);
67 |     }
68 |
69 |
70 | }
71 |
72 | }

```

Líneas 64 a 72: Cierra la conexión (debería estar en un finally).

Resumen para estudiar

- **Singleton:** constructor privado + atributo estático + getInstance() (synchronized para hilos). Se verifica con ==.
- **DAO:** entidad + interfaz + implementación. Factory elige la implementación. El **servicio** coordina varios DAO.
- **Transacción:** setAutoCommit(false) → commit() / rollback() → close(); todas las sentencias en la **misma Connection**.
- **Maven:** pom padre (packaging pom) + módulos hijos con <parent>; un módulo puede depender de otro.
- **NegocioException** = RuntimeException (no controlada); SQLException = controlada.

Cómo se aplica en el parcial de logística

- ConexionDB es el Singleton de conexión; DAOFactory/JdbcDAOFactory es el Factory; las interfaces ChoferDAO, CamionDAO, ViajeDAO son los DAO; el proyecto logistica-dao y el proyecto logistica-web son los dos módulos Maven.
- La transacción de alta de viaje y de alta de chofer sigue exactamente este esquema (con rollback y cierre en finally).

Preguntas típicas

¿Por qué el constructor del Singleton es privado?

Para que nadie pueda crear instancias con new; la única forma de obtener el objeto es getInstance(), que garantiza una sola instancia.

¿Para qué se pasa la Connection por parámetro a los métodos del DAO?

Para que todos los pasos de la transacción usen la misma conexión; si cada método abriera la suya, el commit/rollback no los abarcaría.

¿Qué pasa si falla el paso 3 de la transferencia?

Se lanza una SQLException, se ejecuta rollback() y se deshace el descuento del paso 2: el sistema queda como antes de empezar (atomicidad).